Nimbus Studio

Working with simulations and scientific figures

Nimbus project documentation

18 October 2026

Contents

1 What Studio does and where it stops **1**
- 1.1 The first launch has no storm to display . . . 1
- 1.2 The main working areas . . . 2

2 Configuration, launch and data discovery **2**
- 2.1 A draft must agree with the executable . . . 2
- 2.2 Run status and continuation . . . 3
- 2.3 Nimbus histories and imported histories . . . 3
- 2.4 Health and alignment are part of opening data . . . 3
- 2.5 A practical first inspection . . . 4

3 Reading a plot as a scientific statement **4**
- 3.1 Coordinates come before color . . . 4
- 3.2 Slice, vector and composite views . . . 5
- 3.3 Zoom, native cells and smooth display . . . 5
- 3.4 Profiles, soundings and hodographs . . . 6
- 3.5 Time series, time-height plots and swaths . . . 6
- 3.6 Radar products . . . 7

4 Three-dimensional views **7**
- 4.1 Scientific mode and presentation mode . . . 7
- 4.2 Volume rendering . . . 8
- 4.3 Isosurfaces . . . 8
- 4.4 Streamlines and parcel paths . . . 8
- 4.5 Rendered supercell and coordinated layers . . . 9
- 4.6 Camera, orientation and appearance controls . . . 9

5 How a saved field becomes a screen image **10**
- 5.1 Native reading and the process boundary . . . 10
- 5.2 Workers prepare data without owning the interface . . . 10
- 5.3 Plotly and VTK have different jobs . . . 11
- 5.4 Textures, triangles and shaders . . . 11
- 5.5 Context loss and scene reconstruction . . . 12

6 Frame identity, playback and coherent presentation **12**
- 6.1 Requested, loaded and displayed are distinct . . . 12
- 6.2 Multi-layer scenes commit together . . . 13
- 6.3 Playback is an output-viewing schedule . . . 13

7 Comparison and the research workspace **14**
7.1 Comparing datasets without losing their identities . . . 14
7.2 Research views are explicit analysis recipes . . . 14
7.3 Counts, weights and physical measures . . . 15
7.4 Parcel attribution has a finite information budget . . . 15
7.5 Publishing a research result . . . 15

8 Python and derived fields **16**
8.1 A Python runtime inside the application . . . 16
8.2 Units and coordinate bindings . . . 16
8.3 Full numeric result versus bounded preview . . . 17
8.4 Built-in, expression and external derived fields . . . 17

9 Workers, memory and responsive interaction **17**
9.1 Why more workers can be slower . . . 17
9.2 The I/O memory preference . . . 18
9.3 The different pools of memory . . . 18
9.4 Admission, replacement and retirement . . . 19
9.5 What the resource panel can and cannot certify . . . 19
9.6 ARM64, x86-64 and memory topology . . . 20

10 Exporting figures and recordings **20**
10.1 Choose the product to match the question . . . 20
10.2 Exact-frame recording . . . 21
10.3 Capture must preserve appearance as well as time . . . 21

11 Remote computation and remote workspaces **21**
11.1 Run remotely, inspect locally . . . 21
11.2 Session ownership is deliberate . . . 22
11.3 A remote desktop is a separate feature . . . 22

12 Diagnosing a problem without changing the science **22**

13 Implementation map and verification boundaries **24**
13.1 Where the behavior is implemented . . . 24
13.2 What bounded tests establish . . . 25
13.3 Limits that remain explicit . . . 25

1 What Studio does and where it stops

Nimbus Studio is the desktop workspace for setting up atmospheric experiments, launching the Nimbus executable, inspecting saved fields, comparing datasets and publishing figures. It gives one experiment a common place for its configuration, run status, data sources, plots and analysis recipes. The intent is to make careful experimentation practical on ordinary computers without requiring each researcher to rewrite a model or build a separate visualization application.

The atmospheric solver and Studio are independent programs. The solver advances the equations; Studio displays products the solver has published. A camera move, color change or Python analysis does not advance or modify the simulated state. Closing a plot does not delete the simulation's output. Conversely, a pleasing render is not evidence that its source experiment satisfies a conservation, convergence or scientific-validation requirement.

Nimbus was independently developed from the beginning. CM1 appears in Studio as a read-only dataset and configuration-interchange source, not as the engine behind Nimbus. Similar variable names or familiar controls do not imply identical equations, initial conditions, closures or numerics. Use the mathematics guide for those assumptions and the architecture guide for solver ownership and execution.

This guide follows the ordinary application paths. It describes implemented features and their admission checks, including intentionally unavailable paths. It does not certify every machine, renderer, field combination or scientific interpretation. The terms Ready and Fully Qualified retain the repository's meaning: bounded tests, completed long campaigns and independent scientific review are distinct evidence. A successful image is only one part of that chain.

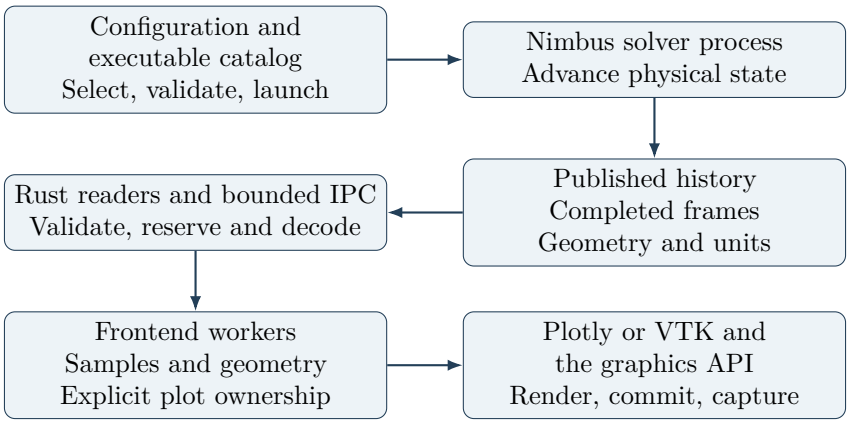


Figure 1: The solver publishes data across a process boundary. Studio renders those data through a separate ownership chain. Imported histories enter at the reader boundary; changing the view does not feed back into the solver.

1.1 The first launch has no storm to display

A fresh checkout and fresh Studio profile contain no pre-run simulations. The Runs view may be empty; the Plots workspace offers **Open data**. These are normal states. Studio can open, show configuration controls and report its preferences without running the solver or downloading scientific runtime tables. The build guide describes a build-and-open-only check for Windows, Linux and macOS. This is the appropriate first check on a new machine.

There are then two ordinary ways to obtain a dataset. Open an existing Nimbus run or supported CM1 history through Open data, or configure and launch a Nimbus experiment. A run directory is the useful unit of Nimbus data: its metadata and completion records give meaning to the field files. Selecting an arbitrary array without that context cannot reconstruct its coordinate system, units, valid domain or simulation time.

The native desktop application provides filesystem access, process launch and native readers. The development browser frontend can exercise rendering and synthetic harnesses, but its mocked native operations are not a replacement for the installed application. Browser rendering evidence and native application evidence should be reported separately.

1.2 The main working areas

Configure holds the experiment draft and its launch controls. **Runs** exposes launched or discovered histories, status and logs. **Plots** opens a data source and builds a viewing workspace. Settings contains local paths, appearance, loading performance and remote connections. Projects and investigation workspaces retain related source and view choices so the user can return to an analysis without treating one transient screen as the whole experiment.

Preferences such as text size, light or dark appearance and which side holds the plot affect the desktop workspace. They are not solver configuration. Similarly, the Loading workers preference controls Studio's field-loading concurrency, whereas the solver's threading and compute-backend settings belong to the run. Keep these two kinds of tuning separate when diagnosing performance.

Saved preferences and recent-dataset entries are conveniences, not archives of the underlying data. Moving or deleting a source can make a saved project unresolvable. Preserve the source history, configuration and any external inputs alongside a project when reproducibility matters. Exported investigation and research packages add explicit recipes and provenance; they do not silently turn every linked source into an embedded simulation archive.

2 Configuration, launch and data discovery

2.1 A draft must agree with the executable

The configuration editor combines a structured form with YAML. Public templates provide starting experiments; they are not certificates for a storm, a resolution or an arbitrary set of physics choices. The complete namelist reference inventories strict-v1 settings, omission defaults, units, aliases and restrictions, including options not exposed as form controls and legacy-reader compatibility.

Studio obtains a scheme catalog from the selected Nimbus executable. The catalog describes its selectors and admission constraints. A UI label alone does not establish that the executable implements the requested combination. Validation and preflight check the actual draft, target, executable identity, required inputs and configuration dependencies before launch. A configuration can contain individually recognizable schemes while their coupling is still rejected.

Scientific tables are separate runtime inputs. Thompson and RRTMG requirements are described in the build guide; opening Studio does not imply that these tables have been installed. Provisioning a canonical table set and checking its identity prevents compiler-dependent reconstruction from silently substituting different input bytes. A missing input should be repaired at its reported path,

not bypassed by changing an unrelated numerical tolerance.

The binary-path preference selects an executable when automatic discovery is not the intended build. The runs-directory preference selects the library root for newly launched histories. Inspect these settings when several checkouts or installed versions coexist: the source tree being edited and the executable being launched are not necessarily the same build.

2.2 Run status and continuation

Launching creates a separate solver process. Studio collects status and log information; run progress and saved-frame availability are different signals. A process may be advancing while no new output frame is complete. Conversely, a finished process can leave useful completed frames even when a later operation failed. Read the reported outcome and logs rather than interpreting the presence of a directory as success.

Checkpoint inspection belongs to continuation, not to plotting. A history frame contains display products; a valid continuation must retain all state required by the selected runtime. Do not assume that being able to plot a time means it is a restart endpoint. The executable's checkpoint and configuration checks remain authoritative when preparing a continuation.

2.3 Nimbus histories and imported histories

Nimbus output readers support the maintained NPY, ZFP and netCDF history paths. NPY remains supported while the required netCDF replacement scope is being completed and qualified. ZFP is a storage codec, not a new coordinate system or a guarantee of losslessness: retain the output's compression provenance when comparing values. Opening a compressed field should not erase how its values were encoded.

A published output hierarchy can provide checksummed levels of detail and bounded native reads. Without such a hierarchy, level zero still has its native read contract; Studio must not invent a published coarse product merely because it can produce a smaller display. Source selection records whether a plot uses a published level, a bounded source window, a reduction or display reconstruction.

CM1 import can inspect history files, a case directory, namelist information and an input sounding. Its compatibility and decision views distinguish direct mechanical mappings from unresolved scientific decisions. Translation can produce a partial draft that still needs interpretation. Nimbus does not install or run CM1, and importing CM1 data does not make Nimbus a CM1 fork.

An adapter records source kind, locator, geometry and availability. Rank matters: a two-dimensional column product cannot enable a three-dimensional renderer just because its field name looks atmospheric. Vector views also require physically matched components. The plot picker therefore uses dataset capabilities rather than presenting every plot as if every file could satisfy it.

2.4 Health and alignment are part of opening data

Dataset health checks expose missing fields, shape or coordinate disagreements, time-axis problems and incomplete products. Treat a missing value as missing, not as calm wind, dry air or zero

precipitation. A finite array value is not by itself proof that the location lies inside the physical domain: halos and boundary closure planes are excluded according to the saved geometry.

For comparisons, inspect the alignment report before interpreting a difference. Time, physical coordinates, units, component meaning and valid-domain support must match the requested operation. A common shape is insufficient: two arrays can have equal dimensions but different origins, spacings, model frames or units. When an operation requires aligned grids, a rejected mismatch is useful feedback, not a reason to strip the metadata.

2.5 A practical first inspection

Open a completed history and start with one scalar Slice. Check its field name, units, saved time, axis labels, slice location and source disclosure. Move to a second completed frame and confirm that both the values and time label change. Then add a vertical profile or cross section through an identifiable feature. Only after those coordinate checks should a volume or composite storm scene be used to explain the feature's three-dimensional appearance.

This sequence makes a dense 3-D view easier to interpret. An opaque volume can hide an interior maximum; a threshold surface omits everything below its chosen level; a derived diagnostic can require fields that were never exported. A simple slice provides a direct way to distinguish a data problem from a camera, transfer-function or feature-selection problem.

3 Reading a plot as a scientific statement

3.1 Coordinates come before color

A field is more than a rectangular array. It has a location on a grid, a physical unit, a domain mask and a saved time. A C-grid velocity component lives on its own faces; a scalar often lives at cell centers. Earth-frame winds, model-frame winds and terrain-normal transport components cannot be exchanged solely because all three have velocity units. Studio's coordinate and vector helpers preserve these distinctions or reject a view that lacks the required geometry.

Cartesian axes can be translated or nonuniform. Terrain-following levels can have different physical heights in each column. Cylindrical geometry includes an angular coordinate. If physical coordinates are unavailable, the application must identify index units rather than attach metres to an index. The source's physical-domain bounds determine what may be drawn and reduced; allocated halos do not become extra atmosphere.

The first two slice indices identify the horizontal directions and the third identifies the vertical direction in the normalized field representation. A plan view holds the vertical index fixed. A vertical section holds a horizontal index fixed. The displayed axes and coordinate metadata, rather than an old internal variable name, establish whether an axis means Cartesian distance, angle, height or model level. The plot labelled **X profile**, for example, is a Cartesian profile along x at fixed y and z; the historical component filename does not make it a radial average.

3.2 Slice, vector and composite views

Slice draws a scalar plane. Changing the slice index requests the corresponding plane; zoom changes the viewport. For window-capable sources these operations can request a bounded portion of the source rather than load the entire three-dimensional field. The source shape, window origin, extent, stride and aggregation explain what was returned. A lower-resolution display must not be reported as native merely because its pixels fill the window.

For terrain data, a horizontal model-level plane remains a model-level view: its height can vary horizontally. The vertical-slice path remaps its one-cell thick bounded window onto separable physical coordinates before Plotly displays it. This makes height meaningful on the vertical axis, but it is a documented remapping operation, not an assertion that the original terrain grid was flat. Retain the corresponding disclosure when exporting or comparing the view.

Vector field draws arrows for a physically matched component family. The in-plane components depend on the selected section. Arrow density and scale affect readability; reducing arrow count does not filter the underlying velocity field into a new scientific result. Check the frame convention and units before interpreting arrow direction as Earth-relative motion. A section cannot borrow an unrelated component solely to make a third arrow coordinate available.

Composite combines a filled scalar plane with contour overlays and available vector context. Each layer must refer to the same admitted source plane and frame. Contour thresholds state an explicit condition on values; color and line width are display choices. If a required overlay is absent or still loading, a composite cannot truthfully claim that all selected layers have finished. An older completed view may be retained while its replacement is prepared, with completion and capture bound to the actual displayed identity.

Deselecting the active field is a valid empty state. It should produce a clear *No data to plot* state and release unused requests, rather than accuse the run of failing to export a field the user no longer requested. This distinction also matters to error handling: an empty selection, a pending read and a genuine missing export are different conditions.

3.3 Zoom, native cells and smooth display

A screen has a finite pixel count; a dataset can have many more cells. Studio therefore separates the source samples, samples admitted for a particular view, and the raster used to draw that view. Zooming can refine a bounded preview, but it cannot recover structures absent from the source. A blocky native-cell plot at extreme zoom is sometimes an honest view of the model's resolution.

Plotly's heatmap smoothing control changes shading only. It is applied to regular, finite heatmaps with suitable linear axes. Nonuniform coordinates, missing values, unsuitable axes and incompatible array shapes retain the original shading. In particular, smoothing must not bridge a missing-data hole or reinterpret a stretched coordinate as uniform. The control discloses when original shading is retained.

For two neighbouring finite samples, linear interpolation has the form

$$q(\xi) = (1-\xi)q_0 + \xi q_1, \qquad 0 \le \xi \le 1. \tag{1}$$

This interpolant is bounded by those samples. It can make their transition look smooth but contributes no new independent measurement between them. Two- and three-dimensional linear

reconstruction apply the corresponding tensor-product weights. Smoothing a threshold surface is a different operation and belongs to the disclosed presentation mode described below.

The canvas also follows display pixel density so labels and graphical elements remain legible on high-density screens. That adjustment is independent of the simulation's cell spacing. A higher-resolution PNG can improve text and edge appearance; it cannot turn a coarse numerical field into a fine-grid result. For publication, preserve units, color limits and sampling disclosure alongside the image rather than treating pixel count as a scientific resolution measure.

3.4 Profiles, soundings and hodographs

Vertical displays a scalar against height at a selected column. **X profile** displays a scalar along the Cartesian x direction at fixed y and z. Missing support creates gaps rather than a fabricated continuous line. Check whether the vertical coordinate is physical height or a declared model-level coordinate; a level number is not an altitude.

Hodograph plots $u(z)$ against $v(z)$. Its two horizontal axes must use compatible velocity units and equal geometric scale for direction and turning to remain interpretable. Height is the parameter along the curve, not a third horizontal wind component. Storm-relative analysis requires an explicit motion estimate; translating a hodograph changes its reference frame, not the saved Earth-relative wind.

Skew-T uses a skewed temperature coordinate and logarithmic pressure representation. Pressure and either temperature or potential temperature are required; additional thermodynamic curves require their corresponding inputs. Do not interpret a missing moisture curve as zero humidity. Sounding plots use a portrait-friendly layout because forcing the diagram into a very wide, short pane distorts the geometry that readers use to judge its slopes.

The plotted environmental profile and a parcel-theory interpretation are also different objects. A sounding's appearance alone does not establish CAPE, parcel origin, entrainment assumptions or a storm's future development. Retain those assumptions in the analysis or script used to derive a diagnostic.

3.5 Time series, time-height plots and swaths

Time-series applies a declared reducer to a field over completed saved frames. Mean, minimum, maximum and other offered reductions answer different questions. A native-cell arithmetic mean is not automatically a mass-weighted or volume-weighted integral, especially on stretched grids. Read the selected reducer and geometry rather than infer a conservation statement from a smooth curve. The horizontal coordinate is saved simulation time, not the wall-clock time required to load the frame.

Time-height retains a vertical level while reducing the other directions, producing a Hovmöller-style diagram. Irregular output times and physical height coordinates must remain visible in its geometry. Large gaps between saved frames are not observations of what happened between them. Height-bin averaging and native model-level reduction should not be confused.

Swath accumulates a selected per-cell maximum or minimum across saved frames. This is a history product: an intense swath location need not correspond to the currently selected frame. Moving-domain data require explicit Earth-frame placement if the intended result is a geographic storm track. A moving grid's array index does not stay over the same Earth location. The swath

implementation tracks source motion where the necessary metadata are available.

These views can read many frames. Their cost may therefore be dominated by storage and reductions rather than graphics. Changing the requested history range or field should cancel obsolete work, while shared useful reads may stay active for other consumers. An analysis assembled from fewer saved frames has less temporal information even if its display interpolates smoothly.

3.6 Radar products

The Radar category uses product presets rather than a separate universal radar solver. It selects supported reflectivity-related views from the available dataset. A simulated reflectivity diagnostic is not itself an observed radar scan, and a color scale does not supply beam geometry or observational effects that the diagnostic did not calculate.

Logarithmic reflectivity requires care when reducing it. Averaging decibels and averaging linear reflectivity are different operations. For values D_i in dBZ, a linear-reflectivity mean is

$$\overline{D} = 10 \log_{10} \left(\frac{1}{n} \sum_{i=1}^{n} 10^{D_i/10} \right). \tag{2}$$

The sampling disclosure distinguishes a bounded linear-Z mean from ordinary cell means and extrema. A radar floor in a three-dimensional view is an additional layer with its own source, reduction and color mapping; it must belong to the same displayed frame as the volume above it.

4 Three-dimensional views

4.1 Scientific mode and presentation mode

Three-dimensional views distinguish **scientific** and **presentation** fidelity. Scientific mode uses resolved source data with declared sampling, interpolation and feature extraction. It does not mean that every source cell is always resident or every pixel is an exact source sample. Bounded subsampling, cell means, extrema reductions and linear reconstruction have distinct provenance because they change what one can infer about peaks.

Presentation mode can add explicitly disclosed surface smoothing or support-constrained subgrid detail. These operations can help explain a scene's shape and continuity. They do not establish new resolved vortices, cloud scales or physical gradients. Changing fidelity is separate from choosing graphics quality, ray-step density or screen resolution. A lower-cost scientific render can still be scientifically honest when its reduction is accurately reported.

Sampling provenance records the source lattice, retained lattice and display lattice independently. It also records coordinate resampling, including linear-to-uniform resampling of stretched axes or physical-volume terrain remapping. In a multi-layer scene, the overall disclosure must account for its coarsest contributing sampling; a native-resolution foreground cannot make a coarse background native.

4.2 Volume rendering

Volume displays a three-dimensional scalar through a color transfer function and an opacity transfer function. Color answers what value was sampled; opacity determines how strongly it contributes along the viewing ray. A bright or solid-looking region is therefore a consequence of both the field and the chosen transfer function. Different camera directions can expose or hide the same interior structure without changing the source.

The renderer samples a ray through a bounded volume texture, respecting validity support and the scene's coordinate transform. Missing cells should not become opaque zero-valued material. Interpolating near missing support needs validity handling, not just a color for NaN. Texture capacity and rendering budgets can require a bounded representation; source disclosure remains attached to it.

Color range, opacity range and any threshold answer different questions. Fixing a color range across frames makes visual comparison easier; recomputing a range for each frame can reveal weak structure but can also make unchanged colors represent different values. Read the colorbar and range policy when interpreting growth or decay. Outlier-resistant display ranges can improve visibility while leaving extrema outside the emphasized range; they do not redefine those extrema.

4.3 Isosurfaces

Isosurface extracts the set where a scalar crosses a chosen threshold, using a marching-cubes family of extraction paths. A threshold surface is a geometric representation of a level set, not a material boundary unless the scientific interpretation justifies that identification. Its shape depends on the threshold, source sampling, interpolation and any declared smoothing.

The preparation pipeline can keep scalar backing and extracted geometry in workers for reuse. Reusing a previous contour is valid only when the current prepared dataset identity matches the backing that produced it. Equal field names, equal dimensions or an apparently familiar frame do not prove that a worker still holds the same bytes. When a source is evicted and prepared again, its new resident identity forces the appropriate rebuild.

Lighting changes should not require rebuilding the level set. They update its material and then the committed appearance descriptor, preserving the source and geometry identity. The capture metadata must describe the lighting actually drawn. An export containing the new pixels with the old shadow or fill settings would be an inconsistent scientific figure recipe.

4.4 Streamlines and parcel paths

Streamlines integrate through a vector field at one saved time. The seed region, direction, integration limits and available support determine the curves. A streamline is tangent to the instantaneous vector field; it is not automatically the trajectory of an air parcel through a time-varying storm. Velocity and vorticity streamline choices require the corresponding coherent vector field and valid coordinate interpretation.

Geometry preparation, vector sampling and seed selection are distinct from tube width, color and lighting. A thinner line is not a weaker wind. Cropping the seed region can reveal the requested feature and reduce work, but omitted seeds do not establish that other flow paths do not exist. Invalid or unsupported locations terminate or exclude integration according to the extraction

contract.

3D paths reads saved Lagrangian trajectory products. It therefore needs trajectory output, not merely a set of instantaneous wind fields. The companion **Vorticity budget** view uses the saved parcel tendency terms. These are history-dependent products and are not per-frame volume renderers. Missing trajectory export should disable the feature clearly rather than silently substitute a streamline.

4.5 Rendered supercell and coordinated layers

Rendered supercell combines condensate volume with available layers such as a reflectivity floor, updraft structure and other offered diagnostics. Its availability starts with a supported cloud or total-condensate field; optional layers have their own input requirements. This is a composite visualization, not an additional microphysics scheme or a reconstruction of unexported fields.

Shared volume composition coordinates geometry, textures and shader evaluation across participating layers. Overlapping volumes must use consistent domain transforms and physical sampling. Independently drawing translucent actors in an arbitrary order can change the image; the composition path therefore owns their joint ordering and framebuffer use. Nested outputs additionally need parent/child spatial ownership so overlapping refinement regions are not counted twice as visual material.

Optional cloud presentation detail and motion cues are declared display operations. They must remain supported by the source cloud and its coordinate frame. A moving presentation is not a newly integrated cloud field. Use scientific mode and a direct slice when a conclusion depends on whether a small structure actually exists in the exported model samples.

4.6 Camera, orientation and appearance controls

The camera has a position, target and orientation relative to the physical scene. The compass, domain box, axes and elevation controls help make this relationship explicit. Physical elevation controls need the loaded grid; a placeholder range before geometry is available must not be interpreted as a valid model height.

Color, background, light direction, shadow depth, fill tint and material properties control appearance. They can improve shape perception but also emphasize features for purely optical reasons. Record them when comparing images. In particular, a new lighting setting must update the appearance receipt after a successful redraw without falsely changing the scientific frame or geometry that the scene contains.

Shared and independent control scopes allow several panes to coordinate some choices while retaining their own views. Camera synchronization does not prove that two datasets align scientifically. Confirm source time, domain, units and sampling before using synchronized views as a comparison.

5 How a saved field becomes a screen image

5.1 Native reading and the process boundary

React and TypeScript own interaction, view state and rendering preparation. Tauri's Rust layer owns native filesystem access, readers and process operations. Large numeric payloads cross the native interface as binary arrays with explicit metadata rather than as enormous JSON lists. The reader validates dimensions, types, bounds and source identities before the frontend treats the payload as a field.

A bounded-window request describes an origin, extent and sampling policy. Returned bytes and source work are separate quantities. A block mean can return one value while visiting many source cells; a compressed representation can need codec scratch in addition to its decoded output. Accounting only for the final array would underestimate the temporary native read footprint.

The native netCDF API is loaded once and retained for the process lifetime, while individual file handles are still closed after their operations. This distinguishes the lifetime of a library's executable code and internal cleanup state from the lifetime of a dataset file. Repeatedly unloading a library that has registered process or thread cleanup callbacks can leave callbacks referring to unloaded code. Keeping the library available does not mean retaining every opened dataset or bypassing the reader's serialization and memory controls.

5.2 Workers prepare data without owning the interface

Web workers perform expensive preparation, derived-field work, contour extraction, streamline integration and other bounded tasks outside the UI thread. The interface can remain responsive while a worker scans a field or builds a mesh. Moving work to a worker does not make it free: memory, transfers, queues and completion synchronization still contribute to latency.

Array buffers can be transferred between owners rather than repeatedly copied. A transfer changes who may use the buffer; the sender cannot continue treating a detached buffer as its live backing. Worker dataset registries therefore record identity and lifetime, not just a pointer or field label. Geometry caches and scalar caches have to agree on the actual resident dataset used by a task.

A shared geometry buffer also needs a successful browser and graphics capability check. That check admits a request to the worker queue once; successful admission must not recursively restart its own capability check. Otherwise an apparently asynchronous operation can repeatedly enqueue promise callbacks, starve the interface and grow memory without performing any useful field preparation. Unsupported sharing uses the maintained transfer path, while an unavailable preparation service is reported to the user.

Changing a field or frame creates a new request generation. A completed old task may no longer be useful when it returns. The result is validated against the current owner before publication, so late work cannot overwrite the newly selected view. Cancellation also retires obsolete resource claims. A cache may retain a deliberately reusable result, but that retention must have an owner and be represented in the relevant accounting.

5.3 Plotly and VTK have different jobs

Plotly handles most two-dimensional figures, profiles, sounding diagrams, history plots, Python numeric previews and research panels. Its canvas and vector elements manage axes, labels, selection and zoom. VTK handles the primary volumetric, surface, streamline and trajectory scenes. It consumes prepared datasets and geometry, constructs actors and mappers, and submits graphics work through the chosen browser graphics API.

These are application-rendering paths. The solver's Vulkan and Metal compute backends do not determine Studio's graphics API. A CPU simulation can be viewed with GPU rendering; a GPU simulation can be viewed on another computer. The desktop webview, driver, graphics capability and workload determine which rendering paths are available.

Volume workloads currently use the WebGL GPU path because the maintained vtk.js WebGPU volume implementation does not provide the same required shading, opacity-normal, scattering and close-camera behavior. Geometry workloads can select WebGPU when adapter discovery and the actual VTK device/context initialization succeed. A preference alone is not proof that the requested API became active. Device or initialization failure is disclosed and uses the maintained WebGL path where available. This rendering fallback is separate from the solver's native-required compute admission contract.

5.4 Textures, triangles and shaders

For a surface or streamline, the CPU or worker prepares vertices and connectivity. The graphics API uploads geometry, transforms vertices through the scene and camera matrices, rasterizes primitives and shades their covered pixels. Surface normals and light directions affect brightness. A bright face can be facing a light rather than carrying a larger field value; the colorbar describes the scalar mapping, not the lighting multiplier.

For a volume, a shader samples one or more scalar textures along each viewing ray. Each sample is mapped to a color and opacity. A simplified front-to-back composition illustrates the ownership of those choices:

$$C_{j+1} = C_j + (1 - A_j)\alpha_j c_j, \tag{3}$$

$$A_{j+1} = A_j + (1 - A_j)\alpha_j. \tag{4}$$

Here c_j is a sample color, α_j its opacity over the current sample segment, and C_j, A_j the accumulated color and opacity. Both the step length and transfer function matter. A physically scaled opacity convention can be expressed as $\alpha = 1-\exp(-\sigma\,\Delta s)$ for an effective attenuation σ and path increment Δs. This explains why changing sampling distance without correcting opacity can change how thick a cloud appears. It is an explanation of rendering, not a replacement for the solver's radiation calculation.

Texture interpolation, scalar validity, lighting, shadow sampling, background composition and output color are separate shader responsibilities. Shadows help depth perception but do not represent the atmospheric heating computed by a radiation scheme. A reflectivity floor is composited with its own color encoding and physical plane. The final framebuffer holds rendered pixels; the native field and its scientific units remain separate data.

5.5 Context loss and scene reconstruction

Graphics contexts can be lost because of driver resets, resource pressure or the browser's management of graphics resources. Once a context is lost, its old textures and framebuffer attachments cannot be assumed usable. Studio's owned render-window construction installs traversal protection and framebuffer ownership together before the VTK object is frozen. Independent patches must not overwrite one another's guards.

The loss path stops invalid traversal; restoration reconstructs the scene through its owners. A restored canvas must reach a completed render before it is reported as a current frame or captured. Merely receiving a context-restored event is not enough. This boundary prevents a stale texture, half-rebuilt layer or previous frame's legend from masquerading as the selected result.

Normal teardown is explicit as well. Cached graphics resources are released while their pipeline owners are valid; mapper and framebuffer deletion completes before the graphics context is deliberately retired. Interactor listeners are removed because a document listener can otherwise keep the entire scene alive. A released application reservation is only an ownership record: it does not prove that the browser or driver has immediately returned every resident byte to the operating system. Lifecycle tests therefore distinguish owned resources, context retirement and measured process memory.

Recovery cannot guarantee that every driver or device can support the requested scene. Persistent failure should remain visible and can require a smaller scene, different graphics preference or application restart. What the implementation must preserve is honest state: it cannot silently label invalid or incomplete pixels as a completed scientific view.

6 Frame identity, playback and coherent presentation

6.1 Requested, loaded and displayed are distinct

Selecting a saved frame starts a request; it does not immediately replace the pixels. A field can finish reading before its geometry is prepared, and a scene can finish preparation before the graphics API draws it. Studio tracks these stages independently. The displayed time should describe the pixels currently visible, with the requested frame shown separately when it differs.

Frame identity includes both the saved step and its publication generation. Reopening a reused directory can produce another run with the same step number. Matching only that number would accept data from the wrong publication. Multi- field plots require every contributing array to agree on the exact publication, not just on shape or step ordinal.

The capture predicate is therefore conceptually

$$\text{capture allowed} = (\text{displayed identity} = \text{requested identity}) \ \wedge \ (\text{render complete}). \tag{5}$$

The implementation additionally guards the active source and operation while capture proceeds. A late change in field, view, source or frame invalidates work that no longer describes the current request.

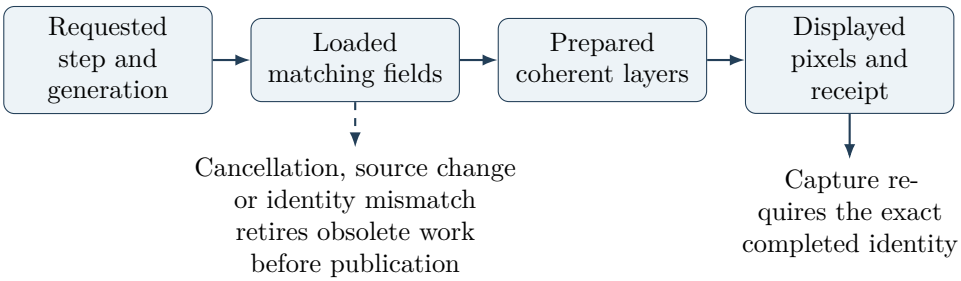


Figure 2: A request is not a displayed frame. The last completed scene may remain visible during preparation; its label and capture state must continue to describe that scene until the new one commits.

6.2 Multi-layer scenes commit together

A cloud volume, updraft surface and radar floor can have different preparation costs. Publishing each as soon as it happens to finish could mix atmospheric times in one picture. A coordinated scene keeps the selected generation and layer identities until the required work is complete. Legends, source disclosure and capture metadata are committed with the scene they describe.

Appearance-only updates have a smaller boundary: they retain the scientific frame and geometry, redraw the changed material, then refresh the appearance part of the capture descriptor. Updating a shadow control must not accidentally claim that a new scientific field was loaded. Conversely, retaining the right frame is not permission to leave the recorded lighting recipe stale.

6.3 Playback is an output-viewing schedule

Playback advances through saved frames, waiting for the participating views' required completion contracts. The requested rate is a target; achievable cadence depends on source I/O, decoding, preparation and rendering. A 60 fps request does not create model states between saved outputs and does not promise 60 newly rendered scientific frames per second on every machine.

Prefetch can prepare future reads while the current frame is displayed. It must remain subordinate to visible work and resource ownership. Leaving a plot or deselecting its field retires its unused prefetch requests. A shared request continues when another visible consumer still needs it. Canceling one pane must not break another pane's valid use of the same field.

The distinction between startup and steady interaction also matters. Creating a large first scene can take longer than changing an already prepared view. Measuring responsiveness before the first completed scientific frame describes startup, not steady playback. Both are useful measurements, but they should be reported as different intervals rather than merged into an unsupported speed claim.

7 Comparison and the research workspace

7.1 Comparing datasets without losing their identities

Comparison views bring scalar slices, composites, vectors, volumes and metric series into a common workspace. Their useful starting point is an explicit alignment decision: which saved times are paired, which physical region is compared, whether units agree and whether the selected operation requires equal grid geometry. Keep each dataset's identity in the resulting publication.

Cellwise subtraction requires a compatible common representation. A difference between two native cells at different physical locations has no automatic interpretation. Ensemble statistics similarly require a declared membership, time alignment and missing-data policy. The number of finite contributing members can be as important as a mean or spread.

For unequal grids, the research Resolution studies view offers native-grid response statistics rather than pretending that arbitrary cells correspond. This can answer how a distribution or threshold volume changes across experiments without silently interpolating every member onto one mesh. It does not replace a controlled convergence study with matching physics, domains, initial conditions and sampling intervals.

7.2 Research views are explicit analysis recipes

The research workspace provides seven linked analysis tabs. It stores the selected source, saved frame, parameters, titles, citations, bookmarks and view ranges in a recipe. Running an analysis produces panels, numerical tables and notes describing what was computed. Changing settings makes an old result non-current until the analysis is run again. Publication waits for the current panels to finish rendering.

These views should be read with their displayed notes and exported tables. Their labels do not establish a complete physical attribution. In particular, an observed feature, a threshold classification and a dynamically closed budget are different scientific claims. The retained recipe states the calculation; independent interpretation still belongs to the researcher.

View	Implemented analysis and important interpretation
Linked sections	A plan locator, a chosen transect, linked scalar sections and profiles. Endpoint and probe choices define what is sampled. A 3-D context panel locates the section; it does not add unmeasured structure.
Environment	Profile overlays, current-minus-reference profiles, soundings and Earth-relative or explicitly storm-relative hodographs. Layer summaries retain finite coverage and the source columns used.
Parcel cohorts	Filters on first and last saved heights, displayed parcel paths, selected-parcel histories and integrals of saved tendency terms. "Origin" means first finite saved position, not an inferred air-mass source.
Cold pools	A declared anomaly threshold, native-level footprint, distributions, linked section and optional history of threshold area and anomaly. Threshold and selected level are part of the definition.
Pressure	Linked pressure and vertical-motion sections. These views do not by themselves perform a pressure-Poisson decomposition or establish force closure.

View	Implemented analysis and important interpretation
Hydrometeors / radar	Height–frequency distributions, conditional profiles, joint distributions and available precipitation histories. Masks, bins, units and finite counts remain explicit.
Resolution studies	Native-grid distributions, responses against declared resolution, physical threshold volumes where geometry supports them, and saved time histories. No unequal-grid cellwise subtraction is invented.

7.3 Counts, weights and physical measures

A histogram of native cells weights each admitted cell once. A physical volume weights each cell by its saved physical volume. A hydrometeor mass additionally needs the correct density carrier and mixing-ratio units. These are different quantities even when their curves look similar. On a stretched grid, many small cells can dominate a count while contributing little physical volume.

For a mixing ratio q per unit dry-air mass, the inventory over cells is

$$M = \sum_i \rho_{d,i}\, V_i\, q_i. \tag{6}$$

Replacing dry density ρ_d with another density convention changes this definition. If exact volumes or the required carrier are unavailable, an analysis cannot claim this inventory merely by summing the array. Research results report unavailable physical measures and retain finite, missing and masked sample counts rather than converting missing data to zeros.

Resolution comparisons admit whole cells by their saved centers within the selected common physical box. Boundary cells are not partially clipped. Nominal cell length, such as $(\Delta x \Delta y \Delta z)^{1/3}$, is descriptive for a stretched mesh rather than a complete characterization of its resolution. Saved times match within an explicit tolerance; the analysis retains actual member times and does not manufacture temporal interpolation.

7.4 Parcel attribution has a finite information budget

The parcel analysis trapezoid-integrates the saved tilting, stretching and baroclinic terms at the saved parcel cadence. Gaps and non-increasing times break the integral; a new finite segment starts from zero. Comparing this integral with observed vorticity change is informative, but three saved terms do not establish a complete vorticity budget. Unsaved forces, mixing terms and behavior between output times are not reconstructed.

Interactive parcel analysis retains at most 64 MiB of saved values. Larger histories need a bounded parcel export for this path. The overview can thin displayed paths and saved samples while retaining the stated cohort definition for its summaries. The notes disclose these display strides so a sparse drawing cannot be mistaken for the complete set of selected parcels.

7.5 Publishing a research result

A research figure package contains PNG panels, a recipe, numerical CSV tables and SHA-256 provenance. The export binds the current result and view settings; changing them during publication

invalidates the in-progress export. Restoring a recipe verifies its source bindings and requires the referenced saved frame. It restores the calculation request, then runs the analysis against the exact available source data; it does not certify a stale image as a new result.

8 Python and derived fields

8.1 A Python runtime inside the application

The Python workspace uses Pyodide in a worker. It is a WebAssembly Python runtime with its own memory and package environment, not the system Python executable. NumPy and Matplotlib are preloaded; the pinned runtime has a package catalog for additional scientific libraries. A package being known to the catalog is different from it already being loaded, cached or usable without network access. Pure-Python package installation can work through micropip; arbitrary native desktop wheels are not interchangeable with Pyodide packages.

Startup is serialized across Python panes to limit concurrent runtime loading. Each active startup attempt has a 30-second deadline, with one fresh-worker retry and a 75-second overall limit including time waiting for another pane. A failed attempt releases its place before retrying so other panes can start. Stop and Restart cancel both queued and active startup. These retries apply only before the interpreter becomes available: Studio never repeats a user's script or silently replaces a running namespace after a code error. Persistent startup failure produces an explicit error instead of leaving the pane loading forever.

Scripts can use bound field arrays, numeric results and Matplotlib figures. The workspace supports cells, saved source, output, completion and traceback presentation. Field references bind data from the selected dataset and frame. The script's operations and assumptions remain the author's responsibility: the application can validate shape and source identity, but it cannot infer that an arbitrary formula has the intended atmospheric meaning.

Stopping an analysis must prevent late results from replacing the active result. Some stop paths retire or reset the worker; preserve useful scripts rather than rely on in-memory Python globals as permanent storage. A persistent workspace recipe or exported script is more reproducible than an undocumented sequence of interactive assignments.

8.2 Units and coordinate bindings

Use `coordinate_units` with `x_coordinates`, `y_coordinates` and `z_coordinates` to inspect the grid's actual axes. The convenience arrays `x_m`, `y_m` and `z_m` contain only physical metre coordinates; an unavailable physical axis is empty. Cylindrical angles are available as `theta_rad`. Index-only geometry remains in index units. Terrain levels remain model-level indices unless the loading path explicitly remaps them to physical height.

The `dataset()` helper preserves these units and supplied scalar coordinates, including nonuniform spacing and translated origins. Inspect them before computing a derivative or integral. A finite-difference expression using one nominal spacing is not generally valid on a nonuniform or mapped grid. Likewise, a gradient of a cell-centered scalar and a face-centered velocity require compatible location and metric conventions.

8.3 Full numeric result versus bounded preview

The Python workspace retains the full supported numeric result for export. A large array plot uses a bounded preview and refines the visible samples when zooming. This separates the amount of data drawn from the array actually computed. Heatmap colors retain the full result's finite range across zooms so that changing the viewport does not silently redefine the colors.

The preview remains limited by the result's real sample spacing. It cannot reconstruct values discarded by a script or increase the scientific resolution of a coarse calculation. Matplotlib figures follow the author's plotting and sampling decisions instead of the automatic numeric-array preview policy.

Numeric-array export and image export are therefore different products. Use numeric export when another analysis needs the values; use a figure when the intended product is the visual explanation. Preserve coordinates, units and script provenance with numeric results so shape alone is not their only meaning.

8.4 Built-in, expression and external derived fields

Derived fields extend the available data without rewriting the solver output. Built-in diagnostics use registered input and unit requirements. Expression fields evaluate an explicit formula over admitted inputs. They inherit the source's time and geometry; combining arrays of the same length is not sufficient if their publications or locations differ.

The external-provider interface adds explicit definitions, input aliases, expected units, output shape policy, source identity and permission requirements. A Python provider runs only after its source digest and permissions are checked. An enabled definition makes it catalog-visible; it does not bypass a missing input, unit disagreement or unavailable runtime. Provider results retain their provenance so a field is not presented as a native solver export.

Native-plugin provider declarations remain unavailable until a compatible host adapter is connected. The presence of their schema or management controls is not a functioning native execution path. This is separate from the local Python snippet discovery feature, which lists scripts for use in the Python workspace.

9 Workers, memory and responsive interaction

9.1 Why more workers can be slower

Loading a field can involve filesystem reads, decompression, copying, metadata validation and reductions. Several workers can overlap independent work, but they also compete for memory bandwidth, cache capacity, storage and decoder scratch. The number of logical CPU cores is therefore not a universal best loading count. It is also unrelated to how many GPU compute queues the solver should use.

On a new native desktop profile, Studio can run a short automatic measurement after the interface becomes idle. It creates a small temporary float32 NPY array, compares cached reads and scanning at candidate worker counts, and removes the temporary fixture afterward. Each choice uses a warmup and three equal-work trials. Automatic selects the smallest count close to the best measured result, avoiding unnecessary concurrency for nearly identical measured throughput.

The measurement is bounded and cancelable. It is available through **Settings → Performance → Measure this computer**, and the automatic measurement can be disabled. The saved profile is reused only when its CPU, core-count and RAM identity matches. An explicitly selected loading count remains the user's choice; the profile applies when the setting is Automatic.

This is a cached NPY read-and-scan measurement, not a sustained disk benchmark, ZFP or netCDF performance campaign, graphics benchmark or solver scaling test. Different working sets, formats and background applications can change the best count. Rerun it while the computer is otherwise idle, then check the actual dataset's responsiveness. The native memory budget can still reduce how many reads run concurrently below the selected worker count.

9.2 The I/O memory preference

I/O memory limit (MiB) controls temporary native field-reading memory. Use zero for an automatic RAM-based policy or a manual value in the range shown by Settings. A MiB is 2^{20} bytes. The permitted manual range begins at 64 MiB; its upper bound depends on available RAM and is capped by the implementation. The active limits panel shows the applied limit, current reserved bytes and waiting reads.

Saving a changed limit applies it without leaving the plot. When concurrent reads temporarily occupy the budget, another read waits for capacity. If one request cannot fit even by itself, it reports the required size rather than waiting indefinitely. Increasing the limit retries applicable budget-failed requests for the same selected field; a user does not need to leave a volume view and return merely to trigger another read. Reducing the limit allows existing reads to finish while new admissions respect the lower limit.

This budget is neither total application RAM nor video memory. Raising it cannot increase a graphics texture limit, remove a contour primitive limit or enlarge Python's address space. It can also increase simultaneous memory pressure, so the useful setting is the one that admits the desired work while leaving room for the rest of the application and operating system.

9.3 The different pools of memory

Studio has several owners with different lifetimes. The resource ledger names native reads, decoded fields, prepared main-thread data, worker-resident backing, worker scratch, handoff buffers, retained geometry and GPU resources. Some buffers move between stages; some are shared by multiple consumers. Accounting uses allocation identity so shared ownership is not mistaken for two physical copies, and a transition is not mistaken for freeing memory that remains live.

The peak working set can be larger than the final picture:

$$B_{\text{peak}} \approx B_{\text{native}} + B_{\text{decoded}} + B_{\text{workers}} + B_{\text{geometry}} + B_{\text{GPU}} + B_{\text{runtime}}. \tag{7}$$

This is an explanatory decomposition, not a direct measurement of whole-process resident memory. Shared allocations, transfers and overlapping lifetimes must be handled by the actual owner ledger. Runtime overhead includes browser and native-library work that cannot be inferred from array sizes alone.

For scale, one $256 \times 256 \times 160$ float32 scalar contains 40 MiB of values. Six such scalar buffers contain 240 MiB before coordinates, copies, validity, scratch, geometry, graphics textures and application

overhead. This arithmetic is useful for planning; it is not a promise that a particular scene uses exactly that amount or that a computer with that much free RAM can render it.

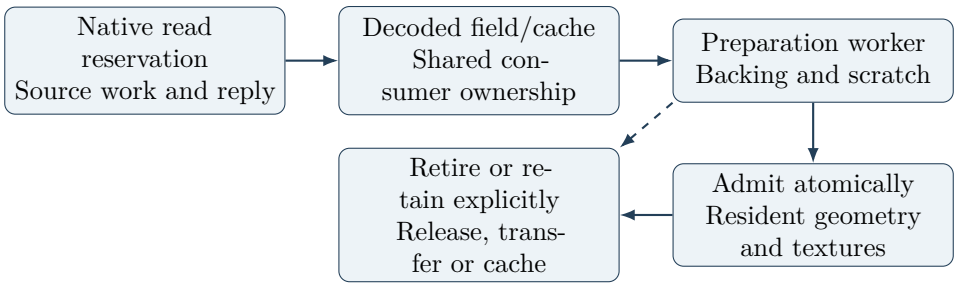


Figure 3: The I/O preference governs the first reservation, not every box. Preparation may publish only after its resident claims are accepted. Cancelled work and deselected layers retire their claims; reusable shared data keeps an explicit owner.

9.4 Admission, replacement and retirement

A preparation task reserves the resources it expects to need. On completion, the owner must transition construction claims to resident claims successfully before publishing the prepared scene. If resident admission fails, the new scene must not become visible with unaccounted backing. This commit boundary keeps resource state and displayed state consistent.

Deselecting a volume field, replacing a preparation request or unmounting a plot retires its inactive bricks and worker resources. Unfinished superseded workers are stopped when their work has no remaining owner. Completed reusable worker backing can remain warm under an explicit cache owner; that is different from retaining every field ever selected. The corresponding error state must also identify the current failed request rather than preserve an obsolete failure from another field.

These rules apply to single and nested volume preparation, and to independently required background reads. A budget increase must reach each applicable owner; retrying only the foreground while leaving its required background failed would leave the feature broken. Shared data stays alive while another consumer needs it, so a drop in one pane's usage need not make every memory counter zero.

9.5 What the resource panel can and cannot certify

The resource panel exposes tracked ownership, reservations, resident commitments and high-water observations. The aggregate visualization-capacity policy remains observe-only unless a matching qualification artifact supplies the required whole-application, GPU-residency and opaque-worker headroom evidence. Separate native I/O limits, queue bounds, texture limits and geometry/output limits still apply.

Observe-only does not mean unlimited memory is safe. It means that the tracked sum is not being advertised as a universally qualified whole-application cap. Browser subprocesses, graphics drivers, native decoder scratch and transient runtime allocations need measurement on the actual packaged application. An observed small-scene success is not a safe upper bound for every dataset.

The packaged lifecycle check verifies scene creation, full-quality captures, explicit object deletion, graphics-context retirement and released canvas backing stores. Its version-3 report records memory

measurements while leaving the capacity campaign pending. Immediate resident-memory growth does not by itself identify a retained scene: garbage collection, browser caches and native graphics reclamation have different lifetimes. Equally, successful object retirement does not establish a sustained memory plateau. Large multi-pane and memory-pressure campaigns must establish that separately.

The same report checks responsiveness during worker operations separately from graphics startup. It retains the delays from both, including first-use texture uploads and shader compilation. A worker check passing does not promise smooth rendering on every device. The graphics-performance campaign remains pending; cold-start delays and sustained interaction performance still require measurement on representative hardware.

9.6 ARM64, x86-64 and memory topology

ARM64 and x86-64 describe instruction sets. Unified and discrete memory describe where processors access storage. Neither distinction alone predicts Studio's performance. An ARM64 application still needs the correct runtime libraries; an x86-64 machine can have integrated graphics; host-visible GPU memory can still have costs unlike a tightly unified memory system.

On a discrete GPU, texture upload can create a separate device allocation and consume interconnect bandwidth. On a unified system, CPU and GPU work still need proper synchronization and ownership, and concurrent access can compete for the same physical bandwidth. JavaScript arrays, worker transfers, native decoder buffers and graphics textures do not become one shared zero-copy allocation merely because the hardware shares memory.

Measure the viewing computer separately from a remote compute node. A remote GPU's memory does not raise Studio's local texture capacity. Compare source identity, field set, sampling, visible layers, worker count and graphics backend when measuring changes. Startup, repeated-frame preparation, steady rendering, capture and file export are separate intervals with different bottlenecks.

10 Exporting figures and recordings

10.1 Choose the product to match the question

A screenshot is a view of pixels. A numeric export preserves values and shape. A recipe records how a view or analysis was requested. A complete reproducible experiment also needs the source data, executable/configuration identity and scientific inputs. These products complement one another; none silently substitutes for all the others.

Plotly's figure controls provide image export for applicable figures. Python numeric results support NPY and bounded CSV export; CSV is deliberately limited because a large text expansion can consume much more memory than the array and cannot represent arbitrary dimensionality without a declared layout. NPY remains a supported numeric export. Use the native application for filesystem save operations and retain meaningful filenames, units and source context.

10.2 Exact-frame recording

The Studio recorder offers GIF, MP4, current-frame PNG and portable HTML viewpoints where the active renderer supports the required capture. The main recording gate currently tracks exact rendered identities for Slice, Vector field, Volume, Isosurface, Streamlines and Rendered supercell. The button is disabled when the selected frame has not completed or when the active view lacks that contract. Comparison, ensemble, investigation and research workspaces have their own publication paths rather than borrowing an unrelated plot's receipt.

Recording selects saved-frame bounds, an output rate and a saved-frame stride. The recorder advances to each selected frame, waits for its exact completed identity and captures it. It does not obtain scientific correctness by sleeping for an arbitrary fixed interval and assuming the renderer caught up. If the source changes, the required frame is unavailable or preparation fails, the capture operation reports the failure instead of silently repeating stale pixels.

GIF encoding is bounded: the current implementation limits encoded output to 32 MiB, 2,000 frames and a 4096×2160 pixel budget per frame. These are application safeguards, not recommended scientific resolutions. Shorter ranges, larger saved-frame strides or smaller figures can make an export fit. MP4 uses an available FFmpeg executable, optionally selected in Settings; it is not provided by choosing a solver GPU backend.

Portable HTML viewpoints package a finite set of captured views for another reader to rotate among. They are not a live arbitrary-camera volume viewer or the original three-dimensional simulation data. They are useful when the recipient needs an accessible visual explanation without installing Studio, provided the discretized viewpoint and source assumptions are retained.

10.3 Capture must preserve appearance as well as time

The frame receipt binds publication and step. A three-dimensional capture also needs its camera, sampling and appearance settings. Lighting-only redraws refresh the committed material description after success while preserving the scientific endpoint. A changed shadow depth, fill tint or other material setting must not export new pixels with a previous lighting recipe.

Legends, floors, annotations and source disclosures are part of the explanatory image. The capture path must wait for their corresponding scene state rather than collecting whichever DOM elements happen to exist when an asynchronous canvas read finishes. Before sharing a figure, verify that its units, saved time, color limits and sampling disclosure describe the claim made in its caption.

11 Remote computation and remote workspaces

11.1 Run remotely, inspect locally

The direct SSH compute workflow launches Nimbus on a named Linux connection. The viewing computer needs OpenSSH. The remote node needs Python 3.8 or newer, the selected Nimbus executable with the current catalog contract and an existing writable work directory. Set up key authentication and host-key trust with the SSH client first; Studio does not provide password or first-host-trust prompts.

Save a connection in **Settings → Compute nodes** using an SSH alias or `user@host`, an absolute executable path and an absolute work directory. Check connection verifies the executable

and catalog. Each configuration begins with **This computer** selected; choosing a saved node makes validation, preflight, checkpoint inspection and launch operate on that target. Removing or changing the selected connection cannot silently redirect a launch locally.

Optional input mappings stage local files into a job's private directory at relative paths used by its YAML. Existing absolute mappings under the configured remote work directory are verified against the local input rather than overwritten. The staged YAML bytes are preserved. This prevents a successful remote launch from quietly using a different file merely because its basename matches an input on the viewing machine.

Confirmed completed, failed and stopped histories are received into the local Runs directory for plotting and continuation. *Receiving results* is a separate phase, with cancellation for the transfer. A received frame is published only after its files pass byte-count and SHA-256 checks. Transfer failure does not erase the remote files. This workflow does not provide live streaming of remote frames into the local viewer.

11.2 Session ownership is deliberate

Direct SSH runs belong to the active Studio session. Keep the viewer awake and connected while a run is active. Quitting or losing the connection retires the remote job through its ownership lease. This is not a detached batch-scheduler submission workflow. An unreachable node produces an explicit error; a stop whose outcome cannot be confirmed is reported as unconfirmed and offers retry when connectivity returns.

A successful local Stop request is not proof that a remote process terminated. The distinction matters when reconnecting or deciding whether another run may use the same resources. Studio reports confirmation state rather than assume that a network interruption implies either continued execution or completed termination.

11.3 A remote desktop is a separate feature

Remote workspaces forward an existing remote browser-accessible service through an owned SSH tunnel and open its local URL. The connection specifies host, port and a validated URL path. Studio checks that the remote service responds; it does not install that service or provision a remote desktop on demand.

Disconnecting this tunnel closes the viewing transport. Jobs managed within the remote desktop remain governed by that remote environment. This differs from the session-owned direct compute workflow above, whose launch and termination leases are part of Studio's process contract. Choose the workflow according to the intended owner of the remote processes, not merely because both use SSH.

12 Diagnosing a problem without changing the science

Start from the observed symptom and identify its owner. An I/O admission error, a missing field, an unavailable graphics capability and a scientific runtime rejection need different repairs. Repeatedly increasing unrelated limits can hide the useful signal while increasing memory pressure.

Observed state	Interpretation and useful next action
Empty Runs or Open data on first launch	Expected without saved simulations. Open a supported history or prepare a validated run. Opening the application does not generate a demonstration storm.
No data to plot after deselection	Expected empty selection. Choose a field; do not edit the output configuration solely because a field is off.
Field or plot unavailable	Inspect available fields, rank, units and required geometry. A 2-D product cannot satisfy a 3-D view, and missing physical coordinates must not be replaced by invented metre axes.
I/O request exceeds limit	Inspect the required bytes and active I/O setting. Apply an admissible larger limit or reduce the source window. Applicable reads retry after the changed budget is applied.
Reads waiting for memory	Other native reads own reservations. Wait, cancel unused requests or adjust concurrency. A queue is different from a single request that can never fit.
3-D scene is blank or faint	Check source support, finite values, opacity, threshold, camera and graphics error state. Use a scalar slice to distinguish absent data from a view that makes them transparent or occluded.
Blocky appearance at zoom	Inspect native spacing and sampling disclosure. Refine a bounded view where supported. Smooth display can improve shading but cannot create source resolution or fill missing support.
Selected time differs from image	Preparation is still running or failed. Read the displayed-frame status; exact capture remains unavailable until the selected publication is drawn.
Recording is disabled	The active view may lack exact-frame reporting, or its frame is not complete. Use its supported figure/package export rather than treat an arbitrary canvas as a verified recorder source.
MP4 encoder unavailable	Check FFmpeg discovery or its explicit path preference. GIF and PNG have different requirements and limits.
Graphics context loss	Allow the owned scene to reconstruct; only a completed restored render is current. Persistent failure needs inspection of scene size, driver/webview and the reported graphics backend.
Remote termination unconfirmed	Restore connectivity and retry Stop. Do not infer process termination from a broken network connection.
Python package import failure	Check the pinned Pyodide package path, network availability and WebAssembly compatibility. A system-Python package installation does not automatically populate the embedded runtime.

A useful defect report includes application/source revision, operating system and architecture, active graphics API, field format, domain shape, selected plot and layers, source/frame identity, exact error text, preferences and a small reproducible input where possible. Record whether the problem occurs on initial load, after changing a field, during playback or during export. These details locate the failing ownership transition more effectively than a lone screenshot of an empty canvas.

13 Implementation map and verification boundaries

13.1 Where the behavior is implemented

The following maintained sources provide the implementation behind this guide. The paths are relative to the repository root. They are starting points for review, not an invitation to bypass the ordinary application path when testing a feature.

Responsibility	Source owners

Application and plot workspace	`ui/src/components/studio/PlotStudio.tsx`
	`ui/src/components/runs/RunDetail.tsx`
	`ui/src/lib/plot/plotRegistry.ts`

Configuration and run admission	`ui/src/components/config/`
	`ui/src/lib/config/`
	`ui/src-tauri/src/commands/config_preflight.rs`
	`ui/src-tauri/src/commands/runner.rs`

Data sources and native readers	`ui/src/lib/dataset/`
	`ui/src/lib/run/runOutputs.ts`
	`ui/src-tauri/src/commands/outputs.rs`
	`ui/src-tauri/src/commands/netcdf_dynamic.rs`

Grid and field interpretation	`ui/src/lib/science/`
	`ui/src/lib/plot/renderFidelity.ts`
	`ui/src/components/plots/PlotSourceDisclosure.tsx`

2-D and 3-D views	`ui/src/components/plots/`
	`ui/src/components/studio/PlotCanvas.tsx`
	`ui/src/lib/vtk/`

Exact-frame and presentation state	`ui/src/lib/run/frameIdentity.ts`
	`ui/src/lib/run/frameReady.ts`
	`ui/src/lib/plot/panePresentationBarrier.ts`
	`ui/src/lib/vtk/coherentSceneGeneration.ts`

Context and framebuffer lifetime	`ui/src/lib/vtk/ownedRenderWindow.ts`
	`ui/src/lib/vtk/vtkContextRenderGuard.ts`
	`ui/src/lib/vtk/vtkTraversalGuard.ts`

Worker, cache and budget owners	`ui/src/lib/core/`
	`ui/src/lib/studio/studioPerformance.ts`
	`ui/src-tauri/src/commands/io_budget.rs`
	`ui/src-tauri/src/commands/worker_profile.rs`

Comparison and research	`ui/src/lib/comparison/`
	`ui/src/lib/research/`
	`ui/src/components/research/`

Python and external providers	`ui/src/lib/platform/pyodideRuntime.ts`
	`ui/src/lib/platform/pyodideWorker.ts`
	`ui/src/lib/studio/pythonWorkspace.ts`
	`ui/src/lib/derived/`

Capture and publication	`ui/src/components/studio/MovieRecorderDialog.tsx`
	`ui/src/lib/run/frameCapture.ts`
	`ui/src/lib/research/publication.ts`
	`ui/src/lib/studio/investigationPublication.ts`

Remote ownership	`ui/src-tauri/src/commands/remote_compute.rs`
	`ui/src-tauri/src/commands/remote_receive.rs`
	`ui/src-tauri/src/commands/remote_workspace.rs`

13.2 What bounded tests establish

Maintained TypeScript unit checks exercise numerical display contracts, coordinates, resource transitions, frame identities, recipe parsing and cancellation. Browser tests exercise real mounted views, controls, rendering, context loss, source changes and capture. Rust tests exercise native readers, filesystem behavior, I/O admission and process ownership. Synthetic arrays and temporary data are generated by the tests under `tests/ui/`; personal simulation datasets are not application assets.

The maintained entry points include `npm run typecheck`, `npm run test:unit`, `npm run typecheck:browser`, `npm run test:browser` and the native Cargo tests. Their prerequisites and build routes are described in the Studio source README and the build guide. A test that needs a simulator, packaged application, browser engine or external integration input must report an unavailable prerequisite rather than convert it into a passing claim.

Chromium software rendering, WebKit, a native desktop webview and a physical GPU are different test environments. Passing one does not automatically establish the other three. High-density display checks and repeated mount/unmount checks exercise additional failure modes but do not measure all driver behavior or prove indefinite resource stability.

13.3 Limits that remain explicit

The application has capability-dependent paths. Missing physical coordinates block metre-scale 3-D geometry; unavailable native provider execution remains unavailable; exact-frame recording is limited to its implemented identity owners; package imports depend on the embedded Python runtime; remote compute does not provide live remote-frame streaming or detached scheduler jobs. Aggregate whole-application capacity requires evidence beyond tracked array sizes. These are concrete boundaries to preserve when extending the product.

Bounded repairs to cancellation, budget retry, capture metadata, resident identity and context restoration strengthen the implemented paths. They do not establish universal performance, all-platform readiness, completed long-running capacity campaigns or independent scientific review. Those statuses must be reported from the applicable evidence rather than inferred from this guide's existence.

When expanding Studio, update the public control, reader/preparation owner, frame and resource contracts, capture/publication metadata, tests and this explanation together. A new visible option is complete only when the ordinary runtime path reaches the intended behavior and its failures remain explicit.